🎨 Five Equations That Paint Impossible Worlds

Create a singularity garden, an interference cathedral, a Möbius aurora, a prime-number nebula, and a recursive dream basin with Python



Zayd Islam

Follow

8 min read

.

2 days ago

Listen
Share
More

Read the previous blog: When Trigonometry Paints

Mathematics does not have to remain trapped inside textbooks. With a few equations, carefully chosen colors, and a little computation, numbers can become alien landscapes, luminous structures, and abstract worlds that appear almost alive.

In this tutorial, we will create five original mathematical artworks. Each experiment explores a different idea:

1. Complex-number domain coloring
2. Wave interference

3. Non-orientable geometry
4. Prime-number distribution
5. Newton's method and fractal basins

All five programs have been tested successfully.

Before We Begin

Install the required packages:

```
pip install numpy matplotlib
```

Each visualization can be run independently in a Python script or Jupyter notebook.

1. The Singularity Garden

The first artwork begins with a complex-valued function:

where

```
z=x+iy.
```

Unlike an ordinary real function, f(z) produces a complex number at every point. A complex number contains two useful visual properties:

and

We map the phase to hue and the magnitude to brightness. Zeros, poles, and rapidly changing phases then become luminous petals, tunnels, and singular structures.

Python code

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.colors import hsv_to_rgb

# Resolution
N = 1000

x = np.linspace(-2.7, 2.7, N)
y = np.linspace(-2.7, 2.7, N)
X, Y = np.meshgrid(x, y)

# Convert the plane into complex numbers
z = X + 1j * Y

# The complex equation
f = (
    np.sin(z**3) + 0.35 * np.cos(5 * z)
) / (
    1 + 0.18 * z**6
)

# Phase controls hue
phase = (np.angle(f) + np.pi) / (2 * np.pi)

# Magnitude controls brightness and saturation
magnitude = np.log1p(np.abs(f))
brightness = 1 - np.exp(-1.7 * magnitude)
saturation = 0.72 + 0.28 * np.tanh(magnitude)

# Build an HSV image and convert it to RGB
hsv = np.dstack((phase, saturation, brightness))
rgb = hsv_to_rgb(hsv)

plt.figure(figsize=(8, 8), facecolor="black")

plt.imshow(
    rgb,
    extent=[x.min(), x.max(), y.min(), y.max()],
    origin="lower"
)

plt.axis("off")
plt.tight_layout(pad=0)
```

```
    plt.savefig(
        "singularity_garden.png",
        dpi=300,
        bbox_inches="tight",
        pad_inches=0,
        facecolor="black"
    )

    plt.show()
```

Output

Press enter or click to view image in full size


Figure 1
(x, y) = (−0.744, −0.725)
[0.958, 0.0127, 0.32]

Why the visual looks alive

The term z^3 imposes threefold rotational behavior, while $\cos(5z)$ introduces a competing fivefold frequency. The denominator creates additional singular behavior.

The artwork therefore emerges from a mathematical disagreement between threefold and fivefold symmetry.

Try changing

```
np.sin(z**3)
```

to:

```
np.sin(z**4)
```

Or replace:

```
0.3 * np.cos(5 * z)
```

with:

```
0.5 * np.cos(7 * z)
```

Each change produces a different mathematical species.

2. The Interference Cathedral

Imagine eight invisible wave sources arranged around a circle. Each source sends waves toward every point in space.

For source j, define the distance:

The complex wave produced by that source is:

The total wave is the sum of all sources:

Finally, the visible intensity is:

When waves arrive in agreement, they reinforce one another. When they arrive out of phase, they cancel. The result resembles a glowing cathedral assembled from light.

Python code

```
import numpy as np
import matplotlib.pyplot as plt

# Image resolution
N = 1100

# Coordinate grid
x = np.linspace(-4.3, 4.3, N)
y = np.linspace(-4.3, 4.3, N)
X, Y = np.meshgrid(x, y)

# Arrange eight wave sources around a circle
angles = np.linspace(
    0,
    2 * np.pi,
    8,
    endpoint=False
)

sources = [
    (
        2.2 * np.cos(angle),
        2.2 * np.sin(angle),
```

```
            index * np.pi / 4
        )
        for index, angle in enumerate(angles)
    ]

    # Initialize the complex wave field
    wave = np.zeros_like(X, dtype=complex)

    wave_number = 15

    # Add the contribution from every wave source
    for source_x, source_y, phase_shift in sources:
        distance = np.hypot(
            X - source_x,
            Y - source_y
        ) + 1e-3

        wave += (
            np.exp(
                1j * (
                    wave_number * distance
                    + phase_shift
                )
            )
            / np.sqrt(distance)
        )

    # Calculate wave intensity
    intensity = np.abs(wave) ** 2

    # Compress extreme intensity values
    intensity = np.log1p(intensity)

    # Create the visualization
    plt.figure(
        figsize=(8, 8),
        facecolor="black"
    )

    plt.imshow(
        intensity,
        cmap="magma",
        origin="lower",
        extent=[
            x.min(),
            x.max(),
            y.min(),
            y.max()
        ],
        vmax=np.percentile(intensity, 99.4)
    )

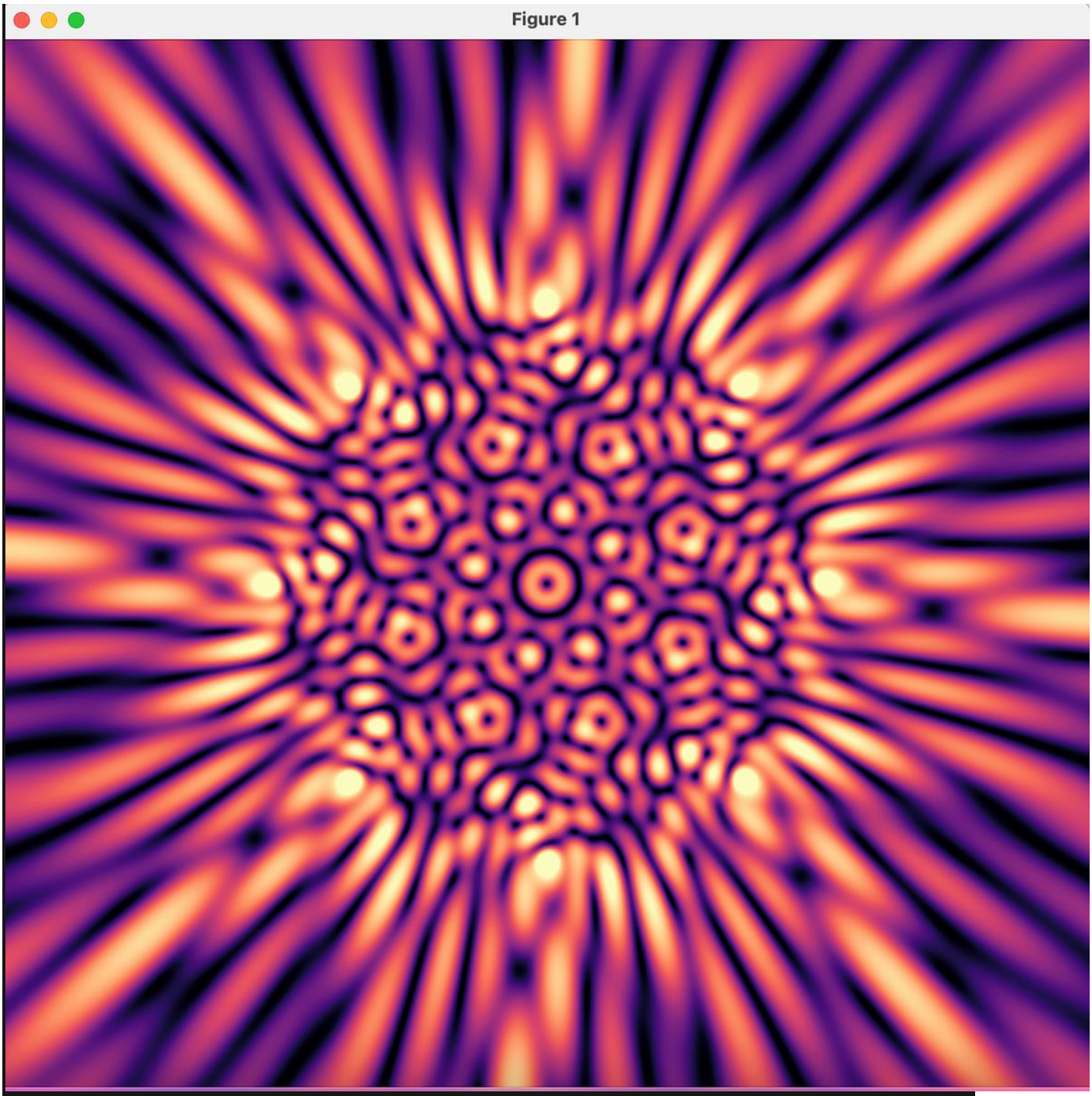
    plt.axis("off")
    plt.tight_layout(pad=0)
```

```
# Save the artwork
plt.savefig(
    "interference_cathedral.png",
    dpi=300,
    bbox_inches="tight",
    pad_inches=0,
    facecolor="black"
)

plt.show()
```

Output:

Press enter or click to view image in full size



The hidden architecture

No explicit flowers, windows, or ornamental patterns were drawn. Python only calculated how the waves interacted.

The apparent architecture is generated entirely by constructive and destructive interference.

Experiment with the wave frequency

Change:

```
wave_number = 15
```

to:

```
wave_number = 25
```

Higher values create finer patterns. Lower values create larger, softer structures.

You can also change the number of sources:

```
angles = np.linspace(
    0,
    2 * np.pi,
    12,
    endpoint=False
)
```

That produces a more intricate radial geometry.

3. The Möbius Aurora

The Möbius strip is a surface with only one side and one boundary.

Its parametric equations are:

with

$$0 \le u \le 2\pi$$

and

$$-0.48 \le v \le 0.48.$$

To transform the surface into an aurora, we place an oscillating color field on it:

The geometry controls the shape. The second equation controls the flowing color.

Python code

```
import numpy as np
import matplotlib.pyplot as plt

# Möbius coordinates
u = np.linspace(0, 2 * np.pi, 1200)
v = np.linspace(-0.48, 0.48, 180)

U, V = np.meshgrid(u, v)

# Möbius strip equations
X = (
    1 + V * np.cos(U / 2)
) * np.cos(U)

Y = (
    1 + V * np.cos(U / 2)
) * np.sin(U)

Z = V * np.sin(U / 2)

# Oscillating color equation
color_field = (
    np.sin(9 * U + 7 * V)
    + np.cos(13 * U - 11 * V)
```

```
) / 2

normalized_colors = (color_field + 1) / 2

fig = plt.figure(
    figsize=(10, 8),
    facecolor="#03040b"
)

ax = fig.add_subplot(
    111,
    projection="3d",
    facecolor="#03040b"
)

ax.plot_surface(
    X,
    Y,
    Z,
    facecolors=plt.cm.twilight_shifted(
        normalized_colors
    ),
    rstride=1,
    cstride=2,
    linewidth=0,
    antialiased=True,
    shade=False
)

ax.view_init(elev=28, azim=38)
ax.set_axis_off()
ax.set_box_aspect((1, 1, 0.45))

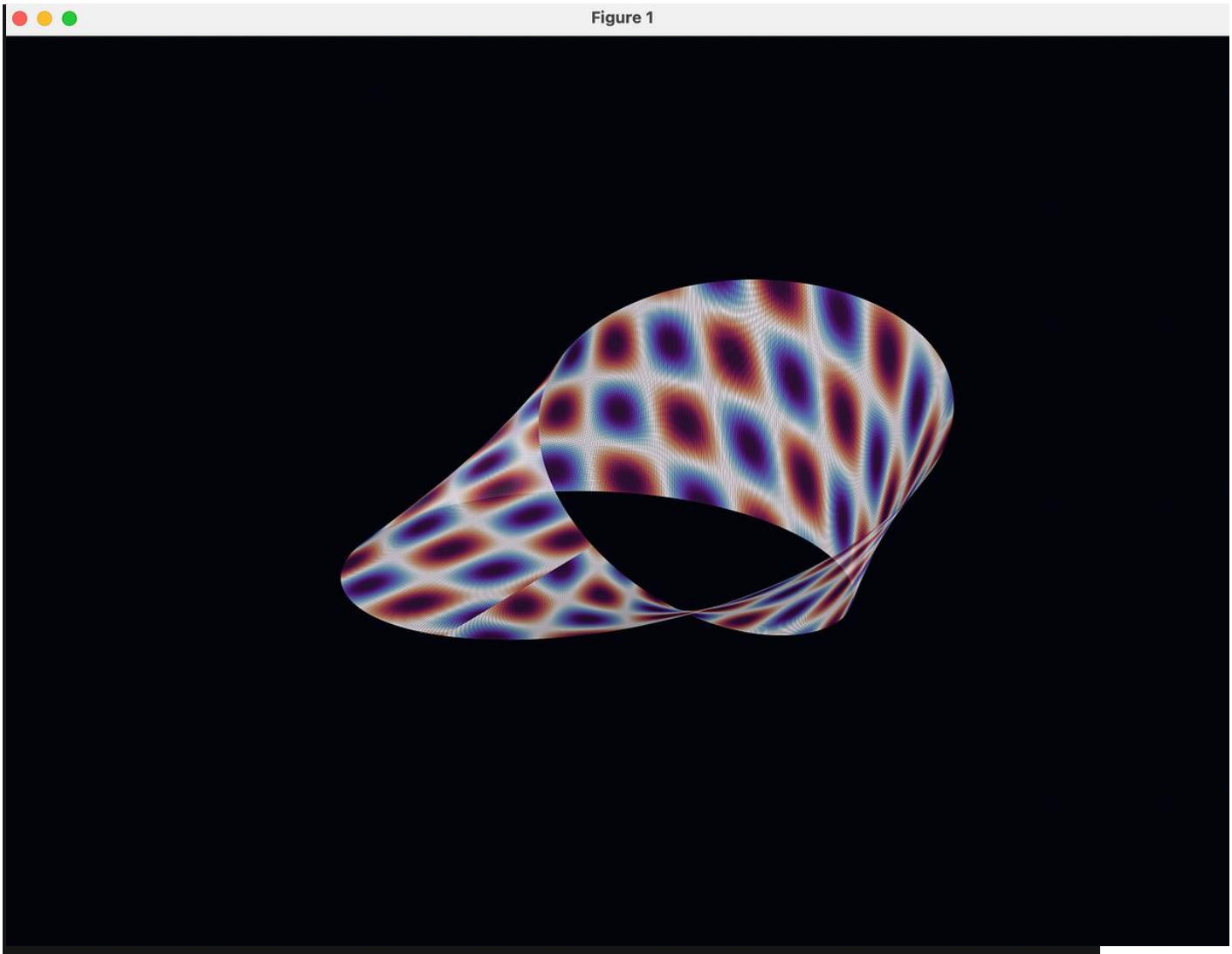
plt.tight_layout(pad=0)

plt.savefig(
    "mobius_aurora.png",
    dpi=300,
    bbox_inches="tight",
    pad_inches=0,
    facecolor="#03040b"
)

plt.show()
```

Output:

Press enter or click to view image in full size



Why the twist matters

If you follow the surface with a finger, you eventually return to the starting position, but from what appears to be the opposite side. The distinction between “inside” and “outside” disappears.

Even the color field is forced to travel through this topological twist.

Create different auroras

Replace the color equation with:

```
color_field = np.sin(
    16 * U + 20 * V + 3 * np.sin(5 * U)
)
```

Or try:

```
color_field = np.cos(
    12 * U * V
) * np.sin(
    8 * U - 6 * V
)
```

The geometry remains the same, but the surface develops a new visual identity.

4. The Prime-Number Nebula

Prime numbers are usually presented as a list:

$$2,3,5,7,11,13,17,\ldots$$

Instead of placing them on a number line, let us send them into polar space.

For every prime p, define:

and

We then convert the polar coordinates into Cartesian coordinates:

Because sqrt{2} is irrational, successive angles do not settle into a simple repeating cycle. The primes become a structured celestial cloud.

Python code

```
import numpy as np
import matplotlib.pyplot as plt

maximum_number = 140000

# Build a Boolean array for the sieve
is_prime = np.ones(
    maximum_number + 1,
    dtype=bool
)

is_prime[:2] = False

# Sieve of Eratosthenes
for candidate in range(
    2,
    int(np.sqrt(maximum_number)) + 1
):
    if is_prime[candidate]:
        is_prime[
            candidate * candidate:
            maximum_number + 1:
            candidate
        ] = False

# Extract the prime numbers
primes = np.arange(
    maximum_number + 1
)[is_prime]

# Polar mapping
theta = primes * np.sqrt(2) * np.pi
radius = np.sqrt(primes)

x = radius * np.cos(theta)
y = radius * np.sin(theta)

# Modular color classification
colors = primes % 29

plt.figure(
    figsize=(9, 9),
    facecolor="#02030a"
)

plt.scatter(
    x,
    y,
    c=colors,
    cmap="turbo",
```

```
        s=0.55,
        alpha=0.78,
        linewidths=0
    )

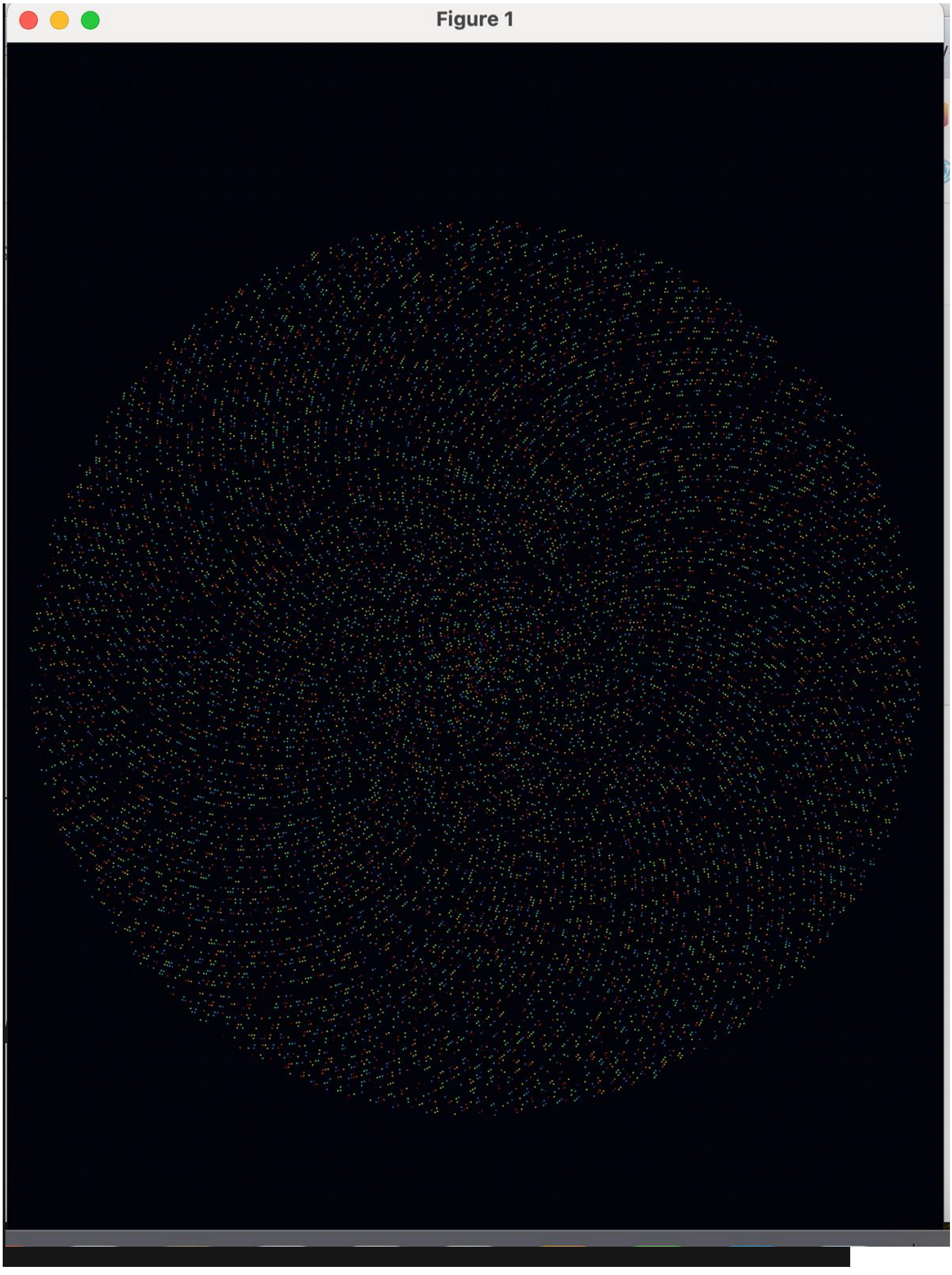
    plt.axis("equal")
    plt.axis("off")
    plt.tight_layout(pad=0)

    plt.savefig(
        "prime_number_nebula.png",
        dpi=300,
        bbox_inches="tight",
        pad_inches=0,
        facecolor="#02030a"
    )

    plt.show()
```

Output:

Press enter or click to view image in full size


Figure 1

Why use sqrt{p} for the radius?

If we used r=p, the outer points would become extremely far apart. Using

compresses the radial growth and distributes the points more evenly across the image.

The color expression

p mod 29

assigns colors according to the remainder produced when a prime is divided by 29. Color therefore represents arithmetic information, not random decoration.

Try a golden-angle variation

Replace:

```
theta = primes * np.sqrt(2) * np.pi
```

with:

```
golden_angle = np.pi * (3 - np.sqrt(5))
theta = primes * golden_angle
```

The entire prime constellation will reorganize itself.

5. The Recursive Dream Basin

Our final artwork uses an equation that repeatedly corrects itself.

Suppose we want to find the solutions of:

$z^7-1=0$.

There are seven complex roots:

Newton's method transforms a current estimate z_n into a better estimate:

For

$f(z)=z^7-1$,

the iteration becomes:

Every pixel begins as a complex number. We repeatedly apply Newton's method and observe which of the seven roots it eventually converges to.

Before iteration, we also bend the starting plane:

This subtle warp turns a traditional Newton fractal into a distorted recursive dreamscape.

Python code

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.colors import hsv_to_rgb

N = 950

x = np.linspace(-1.5, 1.5, N)
y = np.linspace(-1.5, 1.5, N)

X, Y = np.meshgrid(x, y)

# Warp the initial complex plane
z = (
    X + 1j * Y
) * np.exp(
    0.18j * (X**2 - Y**2)
)
```

```
    iteration_count = np.zeros_like(
        X,
        dtype=float
    )

    maximum_iterations = 35
    tolerance = 1e-7

    # Newton iteration for z^7 - 1 = 0
    for iteration in range(maximum_iterations):
        previous_z = z.copy()

        z -= (
            z**7 - 1
        ) / (
            7 * z**6 + 1e-12
        )

        still_moving = (
            np.abs(z - previous_z) > tolerance
        )

        iteration_count[still_moving] = iteration

    # The seven roots of unity
    roots = np.exp(
        2j
        * np.pi
        * np.arange(7)
        / 7
    )

    # Find the nearest root for every pixel
    distances = np.abs(
        z[..., None] - roots
    )

    root_index = np.argmin(
        distances,
        axis=2
    )

    # Root identity and convergence speed control color
    hue = (
        root_index / 7
        + 0.055 * iteration_count
    ) % 1

    saturation = np.full_like(
        hue,
        0.9
    )
```

```
    brightness = (
        0.28
        + 0.72
        * np.exp(
            -iteration_count / 18
        )
    )

    hsv = np.dstack(
        (
            hue,
            saturation,
            brightness
        )
    )

    rgb = hsv_to_rgb(hsv)

    plt.figure(
        figsize=(8, 8),
        facecolor="black"
    )

    plt.imshow(
        rgb,
        origin="lower",
        extent=[-1.5, 1.5, -1.5, 1.5]
    )

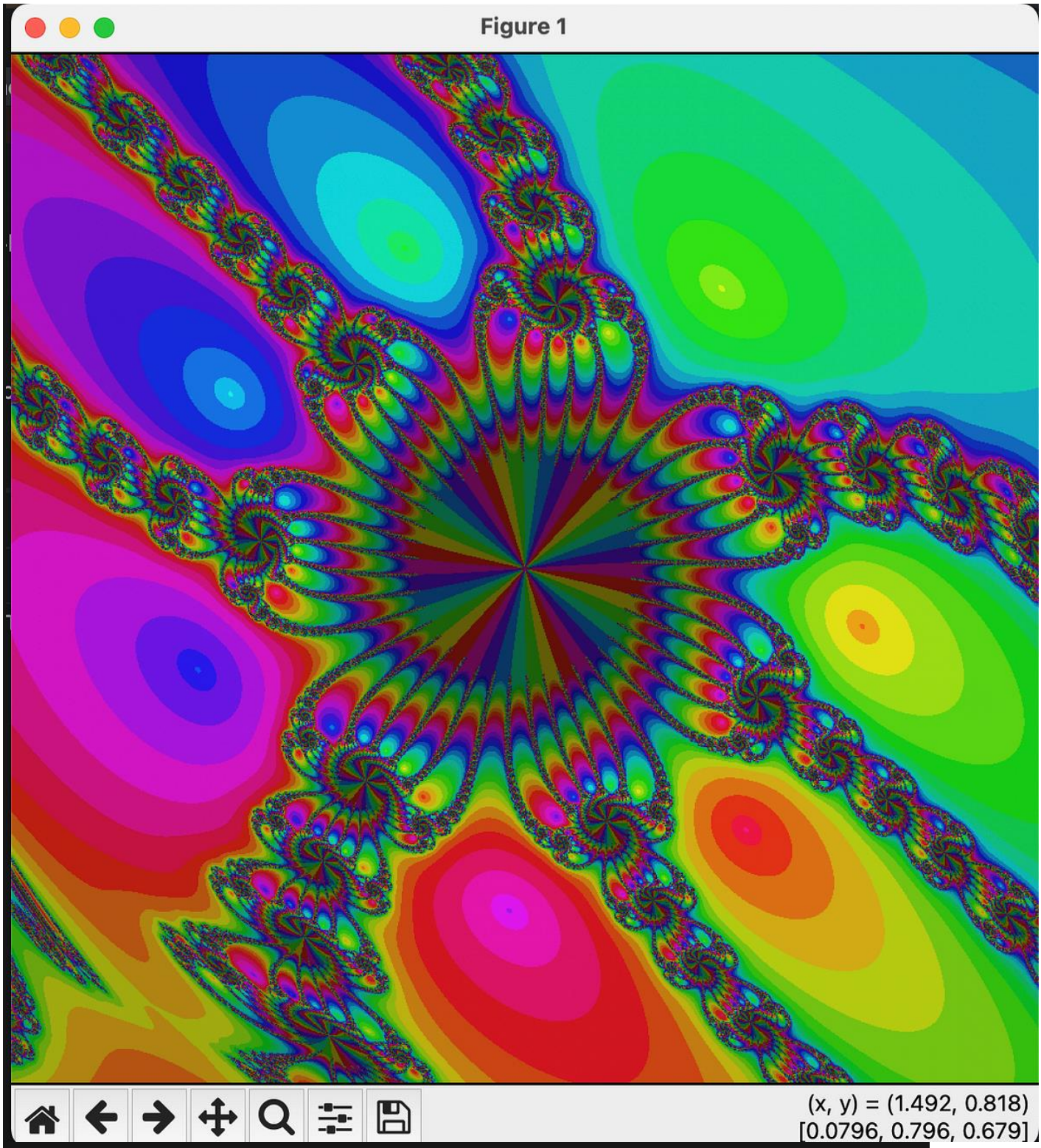
    plt.axis("off")
    plt.tight_layout(pad=0)

    plt.savefig(
        "recursive_dream_basin.png",
        dpi=300,
        bbox_inches="tight",
        pad_inches=0,
        facecolor="black"
    )

    plt.show()
```

Output:

Press enter or click to view image in full size



Reading the fractal

Each major color family represents one of the seven roots.

Broad, smooth regions contain starting points that quickly travel toward the same root. Intricate boundaries contain points whose destinations are extraordinarily sensitive to tiny changes.

Move a starting point by an almost invisible distance, and Newton's method may send it toward a completely different solution.

That sensitivity creates the fractal.

Change the number of attractors

To create a nine-root version, replace:

```
z**7-1
```

with:

```
z**9-1
```

Replace:

```
7 * z**6
```

with:

```
9 * z**6
```

And define the roots using:

```
number_of_roots = 9
roots = np.exp(
    2j
    * np.pi
    * np.arange(number_of_roots)
    / number_of_roots
)
```

Also change:

```
root_index / 7
```

to:

```
root_index / number_of_roots
```

Mathematics Is Not Merely Descriptive

Equations are often treated as tools for describing things that already exist: planetary motion, population growth, electrical signals, or financial behavior.

But equations can also be generative.

A complex function can grow a garden.
A collection of waves can build a cathedral.
A non-orientable surface can carry an aurora.
Prime numbers can become a nebula.
A root-finding algorithm can produce an infinite dreamscape.

None of the programs explicitly draws the final objects. The code only defines mathematical rules and allows every pixel to discover its own fate.

That is the strange power of mathematical art:

We write the law, but the equation reveals the world.

Math

Science

Art